Assignment 2 Solutions

DATABASE SYSTEMS (WEEKS 3 & 4)

Part I: Short Answer / Reasoning Questions

Q1. Counting NULL Phone Numbers & Aggregate Properties

Why the query fails: In SQL, `NULL` represents an unknown or missing value rather than a literal value. The equality operator (`=`) evaluates comparisons against `NULL` as `UNKNOWN` rather than `TRUE` or `FALSE`. Because `WHERE phone_number = NULL` never evaluates to `TRUE`, the `WHERE` clause filters out all rows, resulting in a count of 0.

Corrected Query:

```
SELECT COUNT(*)
FROM Student
WHERE phone_number IS NULL;
```

General Behavior of NULL: SQL operates on three-valued logic (`TRUE`, `FALSE`, `UNKNOWN`). Comparisons with `NULL` yield `UNKNOWN` because it is logically impossible to determine whether an unknown value matches or differs from another. Consequently, most aggregate functions (such as `SUM`, `AVG`, and `COUNT(column)`) omit `NULL` values entirely to prevent missing data from rendering the entire outcome evaluation unknown or distorting metric calculations.

Q2. Redundancy of DISTINCT with GROUP BY

Is the student correct? Yes, the student is correct; the `DISTINCT` keyword is completely redundant in this context.

Justification: The `GROUP BY department` clause already condenses the dataset into unique, distinct department values to calculate the aggregate count for each. Applying `DISTINCT` to the output of a `GROUP BY` forces the query planner to evaluate uniqueness on rows that are already guaranteed to be unique, adding unnecessary processing overhead without altering the result set.

Q3. Row Count Discrepancies in INNER vs. LEFT JOIN

What the difference tells us: This discrepancy indicates that there are records in the `Customer` table that have no corresponding matches in the `Order` table (i.e., customers who have never placed an order).

Scenario causing higher rows in LEFT JOIN: A `LEFT JOIN` preserves all records from the left table (`Customer`), matching them with the right table (`Order`) where keys align and filling unmatched attributes with `NULL` values. If the `LEFT JOIN` yields a higher row count than an `INNER JOIN`, it confirms the existence of these zero-order customer records, which the `INNER JOIN` completely filters out.

Q4. Invalid Non-Aggregated Columns in GROUP BY

Why it is invalid: Standard SQL requires that any column in the `SELECT` clause that is not part of an aggregate function must be explicitly specified within the `GROUP BY` clause. In this query, `employee_name` is a non-aggregated attribute that is absent from the grouping parameters.

Conceptual issue: Allowing this statement would mean grouping multiple individual rows belonging to the same department into a single summary record. Because multiple distinct employee names exist per department, SQL cannot deterministically select which single value to display alongside the departmental average, making the output ambiguous.

Corrected Query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

Q5. Filtering Aggregates (WHERE vs. HAVING)

Identification of error: The query attempts to filter the evaluation of an aggregate function (`AVG(salary)`) using a `WHERE` clause.

Why WHERE cannot be used: In the SQL logical execution pipeline, the `WHERE` clause filters base rows *before* rows are grouped and computed by the `GROUP BY` clause. Because the departmental average salary does not exist at this phase of execution, it cannot be evaluated. Filtering on aggregated expressions must be performed using the `HAVING` clause instead.

Corrected Query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. Subquery Types and Performance

- **(a) Overall average price: Non-correlated subquery.** *Justification:* The overall average price is a single global scalar value independent of any individual item in the outer loop, meaning it only needs to be calculated once during execution.
- **(b) Departmental average salary: Correlated subquery.** *Justification:* The inner average calculation depends directly on the department identifier of the current row being processed in the outer loop, requiring dynamic evaluation per record.

Performance Implication: A non-correlated subquery evaluates once, allowing the query optimizer to cache the scalar result or table expression. A correlated subquery, by contrast, conceptually runs once for every row checked by the outer expression. In large tables, this produces an execution complexity of $O(N^2)$, causing

severe performance degradation unless optimized into an internal join or hash-match structure by the compiler.

Part II: Long Answer Questions

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name, c.email
FROM Customer c
LEFT JOIN Order o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Justification: A `LEFT JOIN` preserves all customer rows regardless of match status, allowing the `WHERE ... IS NULL` condition to precisely capture rows where no corresponding record exists in the `Order` table.

(b) Average order value by restaurant (> 5 orders)

```
SELECT r.restaurant_id, r.name, AVG(order_totals.total_value) AS avg_order_value
FROM Restaurant r
JOIN (
    SELECT o.restaurant_id, o.order_id, SUM(mi.price * oi.quantity) AS total_value
    FROM Order o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.restaurant_id, o.order_id
) order_totals ON r.restaurant_id = order_totals.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(order_totals.order_id) > 5;
```

Justification: This approach utilizes an inner inline view to aggregate line-item values up to a distinct order level first. The outer query then performs an average calculation over these aggregate values per restaurant, using a `HAVING` clause to guarantee that only restaurants exceeding the 5-order threshold are included.

(c) Restaurants exceeding platform-wide average order value

```
WITH RestaurantAverages AS (
    SELECT o.restaurant_id, o.order_id, SUM(mi.price * oi.quantity) AS total_value
    FROM Order o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.restaurant_id, o.order_id
),
RestaurantSummary AS (
    SELECT r.restaurant_id, r.name, AVG(ra.total_value) AS avg_order_value
    FROM Restaurant r
    JOIN RestaurantAverages ra ON r.restaurant_id = ra.restaurant_id
    GROUP BY r.restaurant_id, r.name
    HAVING COUNT(ra.order_id) > 5
)
SELECT restaurant_id, name, avg_order_value
FROM RestaurantSummary
WHERE avg_order_value > (SELECT AVG(total_value) FROM RestaurantAverages);
```

Justification: Common Table Expressions (CTEs) break this problem into isolated, readable layers. A non-correlated subquery is optimal for comparing each restaurant's metric against the platform-wide average since that benchmark remains constant across all evaluations.

(d) Most ordered menu item per restaurant

```
WITH ItemTotals AS (
    SELECT oi.item_id, mi.restaurant_id, mi.item_name, SUM(oi.quantity) AS total_qty
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY oi.item_id, mi.restaurant_id, mi.item_name
),
MaxTotals AS (
    SELECT restaurant_id, MAX(total_qty) AS max_qty
    FROM ItemTotals
    GROUP BY restaurant_id
)
SELECT it.restaurant_id, r.name AS restaurant_name, it.item_name, it.total_qty
FROM ItemTotals it
JOIN MaxTotals mt ON it.restaurant_id = mt.restaurant_id AND it.total_qty =
mt.max_qty
JOIN Restaurant r ON it.restaurant_id = r.restaurant_id;
```

Justification: Isolating the data into structural CTE segments identifies total counts and max values cleanly. A structural limitation of elementary `GROUP BY + MAX` queries is that they cannot pull non-aggregated columns

(such as `item_name`) alongside a computed maximum without either truncating context or causing structural ambiguity; matching back via a multi-column join resolves this gracefully.

Part III: Normalization

Q1. Functional Dependencies

Based on the explicit constraints outlined in the scenario, the following structural dependencies hold true:

1. $\textit{student\_id} \rightarrow \textit{student\_name}$
2. $\textit{course\_id} \rightarrow \textit{course\_name, instructor\_id}$
3. $\textit{instructor\_id} \rightarrow \textit{instructor\_name, instructor\_office}$
4. $\textit{student\_id, course\_id} \rightarrow \textit{grade, enrollment\_date}$

Q2. 1NF Assessment

Yes, the table is in 1NF. Every attribute represents an atomic, single-valued domain, columns maintain distinct naming boundaries, and there are no repeating structural groups or multi-valued entries defined within the system description.

Q3. 2NF Violation & Anomalies

Partial Dependency: The designated primary composite key for the composite schema is $\textit{(student\_id, course\_id)}$. The attributes `student_name` (dependent only on `student_id`) and `course_name` (dependent only on `course_id`) exhibit functional dependencies on only a subset of the primary identifier, violating Second Normal Form.

Insertion Anomaly: A new course cannot be recorded in the system if no student has actively enrolled in it yet, because the `student_id` field forms part of the primary key and cannot accept `NULL` values.

Q4. 3NF Violation & Anomalies

Transitive Dependency: Given that $\textit{course\_id} \rightarrow \textit{instructor\_id}$ and $\textit{instructor\_id} \rightarrow \textit{instructor\_office}$, the office location functionally depends on the course identifier transitively through the instructor's identifier. This structure directly violates Third Normal Form.

Deletion Anomaly: If the final remaining course mapped to a particular instructor is removed from the system, the corresponding data linking that instructor to their specific office room location is permanently lost.

Q5. 3NF Decomposition

To eliminate partial and transitive dependencies, the single un-normalized entity is broken down into four distinct relational schemas:

- **Student** (student_id, student_name)
  Justified by: $\boldsymbol{student\_id \rightarrow student\_name}$
- **Course** (course_id, course_name, instructor_id)
  Justified by: $\boldsymbol{course\_id \rightarrow course\_name, instructor\_id}$
- **Instructor** (instructor_id, instructor_name, instructor_office)
  Justified by: $\boldsymbol{instructor\_id \rightarrow instructor\_name, instructor\_office}$
- **Enrollment** (student_id, course_id, grade, enrollment_date)
  Justified by: $\boldsymbol{student\_id, course\_id \rightarrow grade, enrollment\_date}$

Part IV: Design Justification

In Part II, Query (a) (identifying customers without orders) can be implemented via either a `LEFT JOIN / IS NULL` pattern or an isolated subquery structure using `NOT IN` / `NOT EXISTS`. I opted for the `LEFT JOIN` format due to its execution stability and predictable handling of `NULL` structures. If an inner dataset returns a row containing a missing or `NULL` attribute, a `NOT IN` statement evaluates entirely as unknown and truncates the entire result set. Utilizing join mechanisms offers a resilient execution framework that remains highly readable for database maintenance teams.

In Part III, shifting the schema into 3NF trades off structural purity against query convenience. In the un-normalized layout, rendering a full student roster along with their course names and instructor office rooms requires a straightforward `SELECT *` from a single table. Once decomposed into 3NF, rendering this identical information requires a computationally complex four-table join across `Student`, `Enrollment`, `Course`, and `Instructor`. In high-throughput production environments experiencing intensive read operations, system architects often intentionally re-introduce controlled denormalization to eliminate the CPU overhead of repetitive joins, trading formal normalization rules for improved read latency.